# learn-dev

## Plateforme Web d'apprentissage de la programmation

**Eric Bouchut** – Titre professionnel Développeur Web et Web Mobile (DWWM)

Soutenance – RNCP 37674 – session août 2026

# Introduction

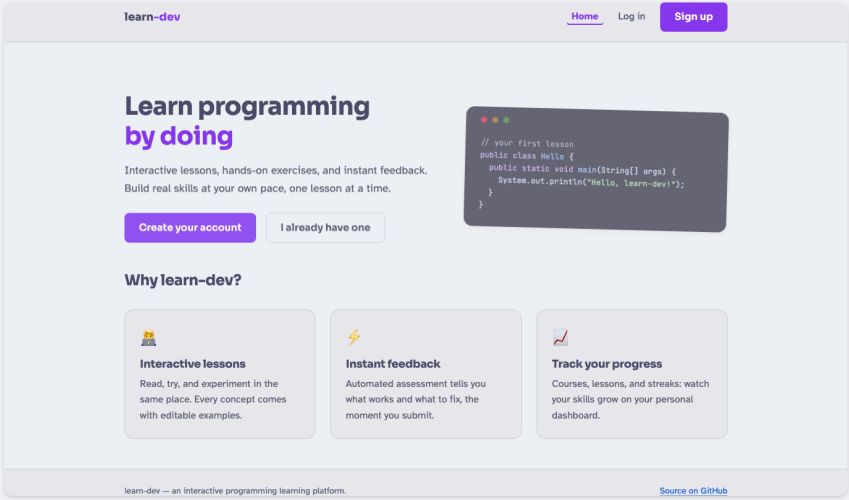
# Le besoin

- **Contexte**: Apprendre à programmer suppose un **parcours structuré** :
  - du contenu organisé,
  - une progression lisible,
  - un espace où un formateur publie et fait évoluer ses cours.
- **Problème** : offrir un environnement multi-rôles simple, **sûr** et **accessible**.
- **Solution** :

> `learn-dev` *est une plateforme Web où des formateurs publient des cours composés de leçons en Markdown, et où des étudiants s'inscrivent et les suivent.*

# Vue d’ensemble de learn-dev

- **Trois rôles** :
  - étudiant ( `STUDENT` ),
  - formateur ( `INSTRUCTOR` ),
  - administrateur ( `ADMIN` ).
- **Étudiant** : catalogue, inscription à des cours, lecture des leçons.
- **Formateur** : création de cours/leçons, cycle de vie de publication, gestion des inscriptions.
- **Administrateur** : gestion des comptes, modération des contenus.



# Conception

# Gestion de projet

- GitHub **Project**,
- **Issues**,
- **ADR**s (*Architecture Decision Records*),
- **Plans** de développement,
- Développement **incrémental** : (authentification, domaine cours, création de contenu, administration),
- Git: **Branche par fonctionnalité**,
- **Conventional Commits**,
- **Pull-Requests**,
- **YAGNI**.

# Cahier des charges et personas

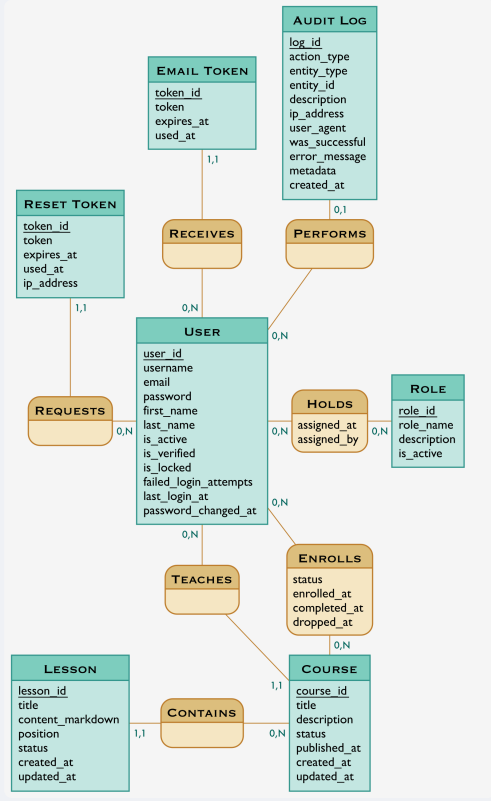
- **Objectifs** : s'inscrire et suivre des cours, publier/gérer des cours, administrer.
- **Personas** :
  - **Léa** (étudiante)
    - s'inscrit et suit des cours, sur mobile comme sur ordinateur.
  - **Marc** (formateur)
    - rédige les leçons de cours qu'il conçoit en Markdown,
    - gère la publication de ses cours,
    - gère l'inscription des étudiants.
  - **Awa** (admin):
    - gère les comptes,
    - modère en toute sécurité.
- **Non fonctionnel** : sécurité, accessibilité RGAA, responsive, tests, CI.

# Base de données (CP5)

# Base de données – MCD

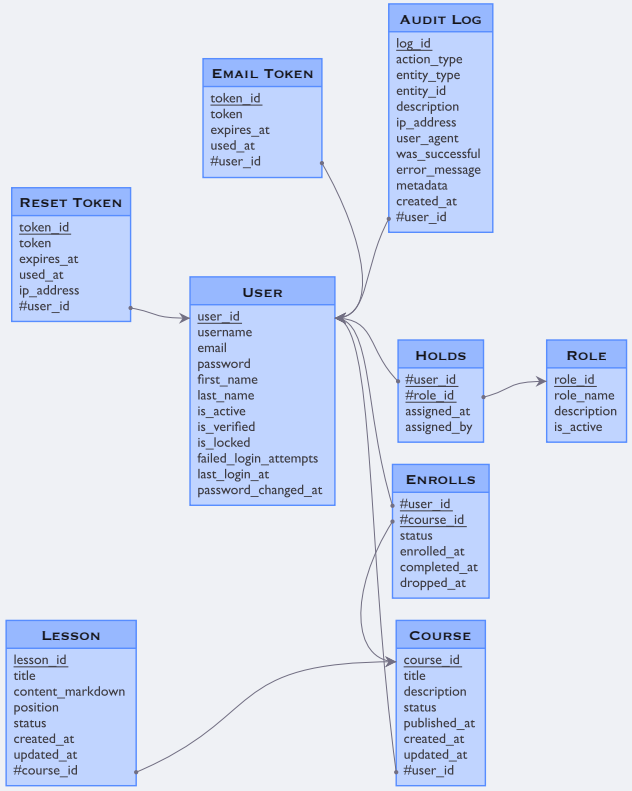
Modèle Conceptuel de Données

- Entités
- Relations
- Cardinalités



# Base de données – MLD (CP5)

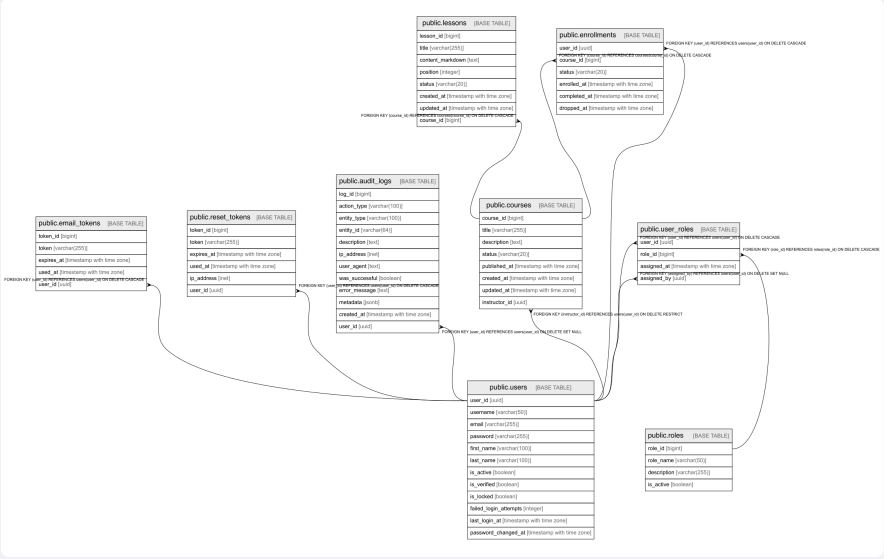
## Modèle Logique de Données (MLD)



# Base de données — MPD

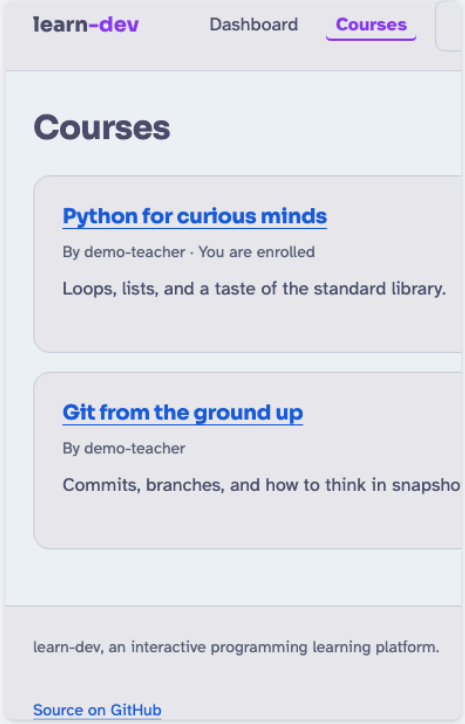
Modèle Physique de Données

- **PostgreSQL 17**, **9 tables**, migrations **Liquibase** appliquées au démarrage.
- **Clé UUID** pour les utilisateurs (anti-IDOR) ; **BIGINT** ailleurs ; clés composites.
- **Contraintes** : `UNIQUE`, `CHECK`, `FK ON DELETE CASCADE` ; Hibernate en `validate`.



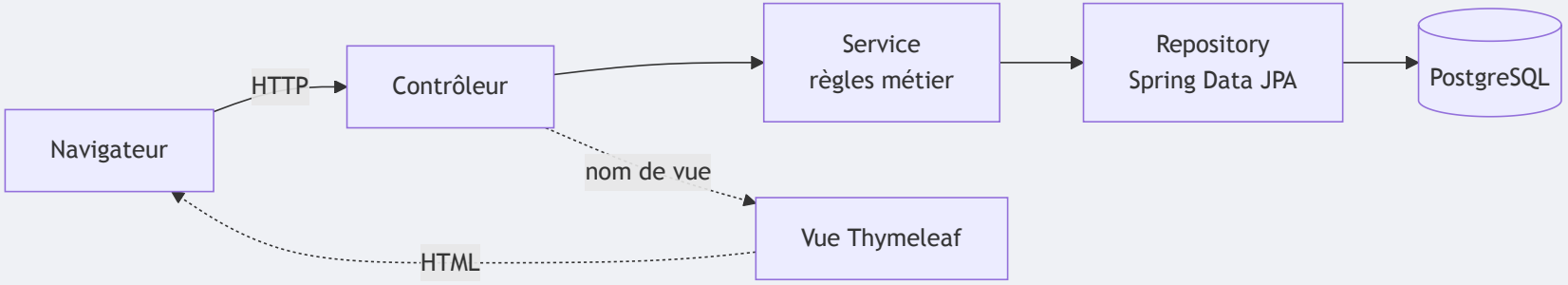
# Maquettage et accessibilité (CP2)

- **Maquettes Figma** + prototypes HTML ; charte, deux thèmes (Catppuccin / Soft Paper).
- **Responsive** : versions Web (bureau) **et Web mobile** (points de rupture en `rem`).
- **Accessibilité RGAA / WCAG AA** : structure sémantique, *skip link*, contrastes calculés (min. 4,73:1), police **Atkinson Hyperlegible**.



# Environnement technique (CP1)

- **Java 21** + **Spring Boot 3.5** (Web MVC, Security, Data JPA, Validation, Mail).
- **Thymeleaf** (vues),
- **commonmark-java**
- **jsoup**
- **PostgreSQL 17** + **Liquibase**.
- **Podman / Docker Compose** (Postgres, Mongo, Mailpit) ; **Maven** ; **Git** ; **CI GitHub Actions**.
- Architecture **en couches** :



# Développement

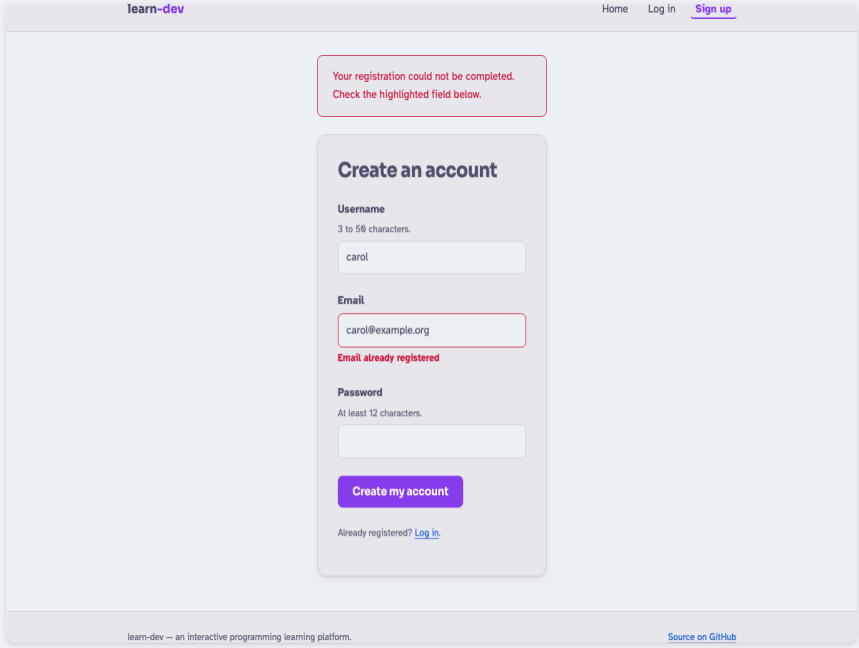
# Deux fonctionnalités CRUD

- **Feature 1 — Inscription d'un utilisateur** (validation, sécurité, jeu d'essai).
- **Feature 2 — Gestion des cours et leçons** (cycle de vie, autorisations).
- Motif commun : **Contrôleur → Service (règles métier) → Repository**, erreurs HTTP explicites (**403 / 404 / 409**).

# Feature 1 — Inscription (CP4, CP7)

- Formulaire **validé** (Bean Validation), **erreurs accessibles**.
- **Unicité** username/e-mail ; motif **Post-Redirect-Get**.

```
public User register(RegisterForm form) {
    if (users.existsByUsername(form.username())) {
        throw new DuplicateUsernameException();
    }
    if (users.existsByEmail(form.email())) {
        throw new DuplicateEmailException();
    }
    // hachage BCrypt, jamais de mot de passe en clair
    user.setPassword(encoder.encode(form.password()));
    return users.saveAndFlush(user);
}
```

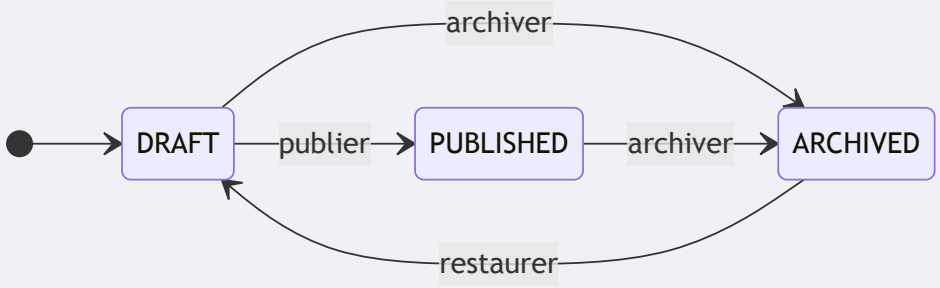


# Jeu d’essai — Inscription

| Scénario | Attendu | Obtenu |
|---|---|---|
| Cas nominal | Compte créé, mot de passe **haché**, rôle `STUDENT` | Conforme |
| E-mail déjà utilisé | Erreur de champ, **aucune** création | Conforme |
| Mot de passe trop court (< 8) | Rejet par la validation | Conforme |
| E-mail invalide | Rejet par la validation | Conforme |

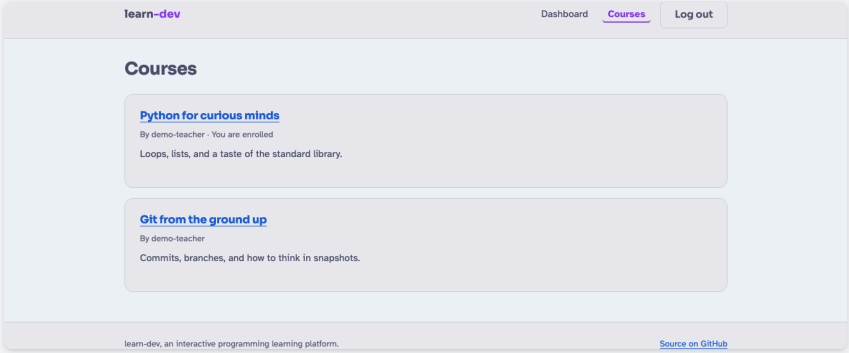
# Feature 2 – Cours et leçons (CP7)

- **Autorisations** : le cours d'un autre formateur → **403** ; contenu invisible → **404** ; transition invalide → **409**.
- Réordonnancement des leçons, gestion des inscrits, **journal d'audit**.



# Front-end : interfaces et responsive (CP3, CP4)

- **21 gabarits Thymeleaf**, HTML5 sémantique, CSS responsive, fragment de mise en page partagé.
- Rendu **dynamique** du Markdown des leçons (converti puis **assaini** contre les XSS).
- Navigation précédent/suivant, affichage conditionnel selon le rôle.



# Accès aux données (CP6)

- **Spring Data JPA** : *repositories* = interfaces, requêtes **dérivées** et **paramétrées** (anti-injection SQL) ; `@Transactional`.
- **NoSQL** : MongoDB **provisionné** en vue d'un stockage de contenu (non encore branché).

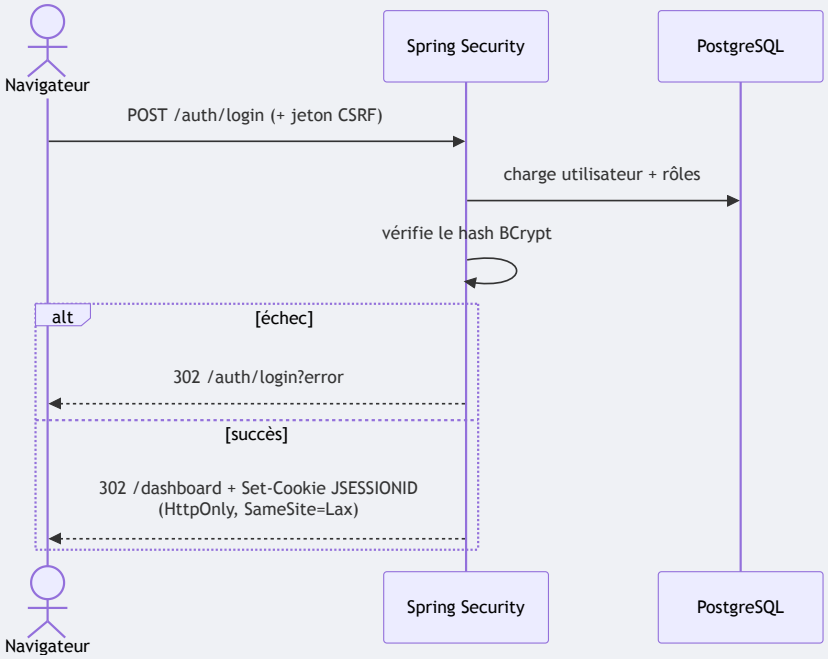
```java
public interface UserRepository extends JpaRepository<User, UUID> {

    @EntityGraph(attributePaths = "roles")   // l'utilisateur ET ses rôles en 1 requête
    Optional<User> findByUsername(String username);

    boolean existsByUsername(String username);
    boolean existsByEmail(String email);
}
```

# Sécurité

- **BCrypt**, **CSRF** activé, **anti-XSS** (jsoup).
- **Anti-IDOR** (UUID), **anti-énumération**, **verrouillage de compte**, **rate-limiting**.
- Jetons **hachés SHA-256**, **journal d'audit**, matrice d'autorisations **testée**.
- Référence **OWASP Top 10** ; veille **Semgrep**.



# Tests

- **64 tests** (pyramide) : unitaires (Mockito), tranche JPA, intégration (MockMvc).
- **PostgreSQL réel** en test via **Testcontainers** (schéma fidèle : UUID, contraintes).
- **Couverture** du code par les tests :
  - mesurée par **JaCoCo**,
  - publiée sur **Codecov**.

# Déploiement

# Conteneurisation et CI (CP8)

- **Image Docker multi-étapes** : build Maven puis exécution **non-root**.
- **Docker Compose** (Postgres / Mongo / Mailpit)
- **Profils** `dev` / `prod`
- **Secrets** via `.env` .
- **Procédure documentée**

```
docker compose up -d        # 1. services (base de données, SMTP local)
./mvnw spring-boot:run      # 2. application ⇒ http://localhost:8080
```

- **CI GitHub Actions** (4 workflows):
  - build,
  - tests,
  - lint,
  - contrôle anti-dérive du schéma.

# Conclusion

# Bilan des compétences

| CP | Compétence | Où |
|---|---|---|
| CP1 | Environnement de travail | Stack, Docker, CI |
| CP2 | Maquetter | Figma, RGAA, responsive |
| CP3 | Interfaces statiques | Thymeleaf, CSS, thèmes |
| CP4 | Interfaces dynamiques | Formulaires, PRG, Markdown |
| CP5 | Base de données | Merise, PostgreSQL, Liquibase |
| CP6 | Accès aux données | Spring Data JPA (+ Mongo) |
| CP7 | Composants métier | Services, règles, erreurs HTTP |
| CP8 | Déploiement | Docker, Compose, CI/CD |

# Difficultés et évolutions

- **Difficultés** :
  - concurrence à l'inscription (contrainte d'unicité captée),
  - sécurité du rendu Markdown (assinissement des entrées pour éviter XSS),
  - un seul `h1` par page (accessibilité).
- **Évolutions** :
  - brancher **MongoDB** (NoSQL),
  - suivre la progression leçon par leçon,
  - API REST pour utiliser de React.

# Merci

**Merci de votre attention.**

Projet : **github.com/ebouchut/learn-dev**

Des questions ?